Comprendre et intégrer Express et EJS dans ChronoSquad

Guide de migration de la branche sprint0

ChronoSquad · 10 octobre 2026 · Référence a6f10b2

Comprendre le rôle de chaque composant, conserver votre interface et construire progressivement un serveur qui fournit les pages et les données.

Parcours de lecture

1 Préambule des changements à réaliser

2 Diagnostic précis de sprint0

3 Comprendre Node.js Express et EJS

4 Architecture cible et contrat des routes

5 Étape 1 préparer la migration

6 Étape 2 démarrer Express et EJS seuls

7 Étape 3 convertir les pages en EJS

8 Étape 4 préparer la persistance et les dépendances métier

9 Étape 5 gérer les comptes et la session

10 Étape 6 brancher les routes et la configuration finale

11 Étape 7 compléter les agendas et événements

12 Étape 8 connecter définitivement le navigateur

13 Étape 9 vérifier le fonctionnement réel

14 Configuration réseau et dépannage

15 Travail restant et ordre conseillé pour l’équipe

16 Repères de lecture et références

1 Préambule des changements à réaliser

Votre interface peut être conservée. Le changement principal consiste à lui donner un véritable serveur applicatif : Express choisira le traitement de chaque requête, EJS produira les pages HTML, et les contrôleurs traiteront les comptes et les agendas en s'appuyant sur les fichiers JSON du serveur.

Aujourd'hui, Node.js reçoit déjà les requêtes HTTP sur le port 3000. Mais il ne fait que lire les fichiers de public/. De son côté, le navigateur simule les opérations métier. Installer Express et renommer les pages en .ejs ne suffira donc pas à rendre la connexion ou la sauvegarde fonctionnelles.

La migration se déroule en deux paliers. Le premier rend le serveur Express et les vues EJS opérationnels. Le second remplace les données simulées par une API réelle avec authentification et persistance. Les exemples de ce guide forment un socle pour les opérations déjà appelées par votre interface. Les fonctions supplémentaires du sujet restent un chantier distinct.

Ce qui change et pourquoi

- Remplacer le serveur de fichiers écrit à la main dans backend/server.js par un serveur HTTP utilisant une application Express. Express gère le routage, les fichiers publics, le corps des requêtes et les réponses.
- Créer backend/app.js pour configurer Express. server.js ne s'occupe plus que de démarrer l'écoute réseau.
- Déplacer les trois pages HTML vers backend/views/ et les convertir en EJS. Ces modèles restent côté serveur ; le navigateur reçoit leur résultat HTML.
- Conserver public/css/ et public/js/. Les styles et les scripts restent téléchargés et exécutés dans le navigateur.
- Séparer les routes de pages, comme /agendas, des routes de données, comme /api/agendas. Une page renvoie du HTML ; l'API renvoie du JSON.
- Remplir les routes, contrôleurs et repositories actuellement vides. Installer les dépendances réellement utilisées.
- Corriger les liens relatifs et les redirections qui pointent vers des fichiers .html.
- Passer USE_MOCK à false seulement quand l'API attendue par l'interface est disponible.
- Fiabiliser les écritures JSON pour plusieurs utilisateurs et corriger le README pour qu'il décrive le code réel.

Ce qui peut être conservé

Le style, les formulaires, les identifiants HTML, la logique du calendrier et l'essentiel des scripts client peuvent rester. Les classes Agenda, Evenement et User constituent une base réutilisable. UserRepository et JsonStore existent déjà, mais demandent des précautions de sérialisation et de concurrence détaillées plus loin.

Ce qui vient du sujet et ce qui relève du choix technique

Le sujet fourni impose un serveur unique pour les pages et l'API, des routes distinctes, le port 3000, le lancement par npm install puis npm start, un accès réseau et la persistance des données. Il autorise les fichiers de sauvegarde. Il ne nomme pas Express ou EJS comme obligation explicite sur sa page 1. Leur utilisation répond ici à votre demande d'appliquer le cours CM06.

Le CM06 présente Express 5, les contrôleurs, les vues EJS, puis une authentification JWT. Votre conception prévoit express-session et votre classe User importe argon2. Ce guide conserve ces deux choix du projet

pour éviter une réécriture inutile. La partie Express/EJS suit le cours ; la gestion de session est une adaptation identifiée, pas une reproduction de son exemple JWT.

2 Diagnostic précis de sprint0

Référence examinée : branche locale sprint0, commit a6f10b2. L'analyse porte sur les fichiers présents le 10 octobre 2026, sans mise à jour du dépôt distant. Aucun fichier applicatif n'a été modifié pour produire ce guide.

Serveur et dépendances

backend/server.js importe http, fs, path et fileURLToPath. Il crée un serveur avec http.createServer, transforme / en /index.html, puis lit un fichier sous public/. La table mimeTypes choisit le type de réponse. Il n'existe aucun traitement des routes /api, aucune lecture de corps JSON, aucune session et aucun rendu EJS.

package.json contient déjà "type": "module" et le script "start": "node backend/server.js". Ces choix sont bons et peuvent rester. Il ne déclare aucune dépendance. Aucun package-lock.json n'est présent parmi les fichiers examinés. Express, EJS et argon2 ne sont donc pas installables à partir de ce manifeste.

Éléments déjà écrits et éléments vides

- Écrits : backend/models/Agenda.js, Evenement.js, User.js ; backend/storage/JsonStore.js ; backend/repositories/UserRepository.js.
- Vides : les quatre fichiers backend/controllers/*Controller.js ; backend/routes/Routes.js ; backend/middleware/RequireAuth.js ; AgendaRepository.js ; EvenementRepository.js.
- Vides également : AgendaTest.js, EvenementTest.js et UserTest.js. Leur présence ne constitue pas une couverture de tests.
- Absents : backend/app.js, backend/views/, configuration EJS et dépendances Express.

Pourquoi le prototype paraît déjà fonctionner

public/js/api.js définit USE_MOCK = true. L'objet mockApi utilise localStorage pour les comptes, agendas et événements, et sessionStorage pour la session simulée. Les mots de passe du mock sont conservés tels quels. Ces données sont liées au navigateur et à son origine ; elles ne sont pas partagées entre les ordinateurs.

L'objet realApi existe déjà dans ce même fichier. Il sait envoyer des requêtes HTTP, mais le serveur actuel ne sait pas les traiter. Il faut implémenter les URL exactes qu'il utilise. Il ne faut pas reprendre aveuglément le tableau de conception : celui-ci indique aussi /api/register et /api/login, qui ne correspondent pas aux URL de realApi.

Écarts utiles à corriger

Le README annonce Express, express-session et bcryptjs, ainsi qu'une arborescence server.js et src/ qui ne correspond pas au dépôt. User utilise en réalité argon2. JsonStore.save écrit directement le fichier alors que la conception annonce une écriture temporaire suivie de rename. JsonStore.load ne crée pas les fichiers manquants.

Les données JSON contiennent déjà des exemples ; ce ne sont pas des tableaux vides à réinitialiser systématiquement. Dans users.json, les valeurs actuelles de passwordHash sont des mots de passe de

démonstration en clair, pas des hashes Argon2. Ces comptes ne peuvent donc pas être vérifiés tels quels par User.verifyPassword. Créez un nouveau compte de test par la future API pour valider l'authentification. Pour conserver les comptes de démonstration et leurs agendas liés, prévoir une migration ponctuelle de ces valeurs vers Argon2 en conservant les identifiants ; ne pas tenter de les rehacher automatiquement à chaque démarrage.

L'interface sélectionne actuellement un seul agenda pour la vue semaine, via selectedAgendaId. La sélection de plusieurs agendas dans la création d'une tâche produit plusieurs créations ; elle ne prouve pas un affichage simultané de plusieurs agendas. La modification et la suppression des rendez-vous ne figurent pas dans realApi. Ces écarts au sujet ne seront pas réglés par EJS.

3 Comprendre Node.js Express et EJS

Node.js exécute votre JavaScript côté serveur

Node.js est l'environnement qui lance backend/server.js. Il peut lire les fichiers JSON, ouvrir un port réseau et exécuter les règles métier. Le navigateur exécute, lui, public/js/agendas.js et dispose du DOM, de document et de window. Ce sont deux contextes différents, même si le langage est JavaScript dans les deux cas.

Un import de node:fs ne fonctionne pas dans vos scripts de navigateur. Inversement, document.getElementById n'a pas sa place dans un contrôleur Node. Les deux côtés communiquent par des requêtes HTTP et des réponses HTML ou JSON.

npm gère les paquets et les scripts du projet. npm install installe les dépendances ; npm start exécute le script start. Node maintient ensuite le processus actif tant que le serveur écoute. Fermer le terminal ou utiliser Ctrl+C arrête le serveur.

Express organise les requêtes

Express est une bibliothèque exécutée par Node. express() crée une application qui peut être passée à createServer(app), comme dans le CM06. Une route associe une méthode HTTP, un chemin et une fonction de traitement.

```
app.get('/bonjour', (req, res) => {
  res.send('Bonjour');
});
```

GET /bonjour appelle cette fonction. req décrit la demande du client ; res permet de répondre. GET lit généralement une ressource, POST crée ou déclenche une action, PUT remplace et DELETE supprime. Une URL sans la bonne méthode ne désigne pas forcément la même opération.

Un middleware reçoit req, res et next. Il peut préparer des informations, répondre, ou appeler next() pour continuer. Un middleware qui ne répond pas et n'appelle pas next laisse la requête en attente. L'ordre de déclaration est donc déterminant.

EJS fabrique du HTML sur le serveur

EJS est un moteur de modèles. Un fichier .ejs ressemble à du HTML, avec des expressions et des instructions JavaScript. Express lui transmet les données avec res.render. EJS calcule le HTML, puis le serveur envoie ce résultat au navigateur.

```
res.render('agendas', {
  title: 'Mes agendas',
  user: { username: 'Alice' }
});
```

```
<h1><%= title %></h1>
<p>Bonjour <%= user.username %></p>
```

Le navigateur reçoit ici un h1 et un paragraphe ordinaires. Il ne reçoit pas les instructions EJS. Renommer un fichier sans le faire passer par res.render ne déclenche aucun rendu.

Le parcours complet d'une action

1. Le navigateur demande GET /agendas avec son cookie de session.
2. Express retrouve l'utilisateur et autorise ou refuse l'accès à la page.
3. Le contrôleur demande res.render('agendas', données).
4. EJS génère le HTML ; le navigateur l'affiche et demande /css/style.css et les scripts.
5. agendas.js appelle api.getAgendas(), qui envoie GET /api/agendas.
6. Express appelle le contrôleur d'API ; celui-ci lit les données par le repository.
7. L'API renvoie un tableau JSON et agendas.js actualise le DOM.

EJS ne rafraîchit pas automatiquement une page déjà ouverte. Vos scripts et leurs appels fetch restent utiles. Cette approche mixte correspond bien à l'interface existante.

4 Architecture cible et contrat des routes

Répartition des fichiers

```
package.json
package-lock.json                 créé par npm
backend/
  server.js                       démarre le serveur
  app.js                          configure Express
  routes/
    PageRoutes.js                 pages HTML
    Routes.js                     API JSON
  middleware/RequireAuth.js       utilisateur et protections
  controllers/
    UserController.js
    AuthController.js
    AgendaController.js
    EvenementController.js
  repositories/                   accès aux enregistrements
  models/                         règles et objets métier
  storage/JsonStore.js             lecture et écriture JSON
  views/
    login.ejs
    register.ejs
    agendas.ejs
    error.ejs

    partials/head.ejs
public/
  css/style.css
  js/api.js
  js/login.js
  js/register.js
  js/agendas.js
data/
  users.json
  agendas.json
  evenements.json
```

Les vues ne doivent pas être dans public/. Le dossier data/ et les sources backend ne doivent jamais être servis comme fichiers publics. Une route n'est pas obligatoirement le chemin physique d'un fichier : /agendas est une adresse Web ; backend/views/agendas.ejs est un fichier interne.

Routes de pages proposées

- GET / : page login.ejs, avec les scripts de connexion.
- GET /register : page register.ejs.
- GET /agendas : page agendas.ejs, réservée à un utilisateur connecté.
- Une URL inconnue hors /api : erreur HTML 404.

Routes API à implémenter pour le client actuel

- POST /api/users : reçoit { username, password }, crée le compte, répond 201 avec l'utilisateur public. Ne connecte pas automatiquement : register.js retourne à la connexion.
- GET /api/users/me : répond 200 avec l'utilisateur connecté ou 401.
- POST /api/auth/login : reçoit { username, password }, ouvre la session, répond 200 avec l'utilisateur public ; 401 si les identifiants sont invalides.
- POST /api/auth/logout : ferme la session, répond 204 sans corps.
- GET /api/agendas : répond 200 avec un tableau d'agendas autorisés.
- GET /api/agendas/:id : répond 200 avec l'agenda autorisé, sinon 404.
- POST /api/agendas : reçoit { name, color }, répond 201 avec l'agenda créé.
- GET /api/evenements : répond 200 avec les événements des agendas autorisés.
- POST /api/agendas/:agendaId/evenements : reçoit { title, start, end, location, description }, répond 201 avec l'événement.

Toutes les routes d'agendas et d'événements nécessitent une session. Un utilisateur public possède id, username et createdAt, jamais passwordHash. Les listes sont des tableaux directs, pas des objets { agendas: [...] }, afin de respecter agendas.js. Les erreurs prennent la forme { error: 'Message' }, comprise par request() dans api.js.

Savoir où lire une valeur

req.params.agendaId vient de /agendas/:agendaId/evenements. req.query vient de la partie ?cle=valeur. req.body contient le corps analysé par express.json() ou express.urlencoded(). L'identité vient de la session vérifiée par le serveur, jamais d'un ownerId librement envoyé par le client.

5 Étape 1 préparer la migration

Exécuter les commandes depuis la racine du projet. Vérifier et sauvegarder son travail avant de changer de branche. Ne pas écraser les modifications d'un autre membre de l'équipe.

```
git status
git switch sprint0
git switch -c migration-express-ejs
node --version
npm --version
npm install express@5 ejs
```

Utiliser une version LTS maintenue de Node.js ; Node 24 LTS est une base adaptée à ce guide. Le minimum technique d'Express 5 ne doit pas être confondu avec une recommandation d'utiliser une ancienne version de Node.

Ne pas refaire npm init et ne pas supprimer package.json. npm install ajoute les dépendances et crée package-lock.json. Versionner les deux pour partager le même socle. Le script start existant est déjà correct ; on peut ajouter le script de développement suivant dans scripts sans supprimer start.

```
"scripts": {
  "start": "node backend/server.js",
  "dev": "node --watch backend/server.js"
}
```

Dans .gitignore, ajouter node_modules/, .env et les fichiers temporaires de développement. Ne pas ignorer package-lock.json. Le guide ne nécessite ni React, ni Vite, ni second serveur, ni compilation front-end.

Le mode --watch est optionnel. Après une modification de configuration, arrêter puis relancer npm start reste la méthode la plus simple. Une modification EJS est visible au prochain rendu en développement ; le navigateur peut nécessiter un rechargement pour CSS et JavaScript.

6 Étape 2 démarrer Express et EJS seuls

Ce premier palier est volontairement indépendant des comptes et des repositories. Il permet de vérifier les chemins, le rendu EJS et les fichiers statiques avant d'ajouter l'API.

Remplacer backend/server.js

```
import { createServer } from 'node:http';
import { app } from './app.js';

const PORT = 3000;
const server = createServer(app);

server.on('error', (error) => {
  console.error('Démarrage impossible :', error.message);
  process.exitCode = 1;
});

server.listen(PORT, '0.0.0.0', () => {
  console.log(`ChronoSquad : http://localhost:${PORT}`);
});
```

createServer(app) reprend la séparation du CM06 pages 30 et 31. app.listen(3000) serait une autre écriture possible, mais il faut choisir une seule méthode : ne pas créer deux serveurs qui écoutent le même port. Le port reste fixé à 3000 pour le rendu demandé par le sujet.

Créer backend/app.js pour ce premier palier

```
import express from 'express';
import { fileURLToPath } from 'node:url';

export const app = express();
const localPath = (value) =>
  fileURLToPath(new URL(value, import.meta.url));

app.set('views', localPath('./views/'));
app.set('view engine', 'ejs');

app.use('/css', express.static(localPath('../public/css/')));
app.use('/js', express.static(localPath('../public/js/')));

app.get('/', (req, res) => {
  res.render('login', { title: 'Connexion' });
});
app.get('/register', (req, res) => {
  res.render('register', { title: 'Créer un compte' });
});
app.get('/agendas', (req, res) => {
  res.render('agendas', { title: 'Mes agendas', user: null });
});

app.use((req, res) => {
  res.status(404).render('error', {
    title: 'Page introuvable', status: 404,
    message: 'Cette page n’existe pas.'
  });
});
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);
  console.error(err);
  res.status(500).send('Erreur interne du serveur');
});
```

Les chemins sont calculés depuis backend/app.js, pas depuis le terminal. ../public/css/ remonte à la racine, tandis que ./views/ reste dans backend/. Aucun import explicite de ejs n’est nécessaire ici : Express charge le moteur configuré.

On monte uniquement /css et /js afin qu’un ancien public/index.html ne prenne pas la priorité sur GET /. Si vous ajoutez des images, montez aussi /images vers public/images/. Ne remontez pas toute la racine du projet en statique.

À ce stade /agendas n’est pas encore protégée et les scripts peuvent rester en mode mock pour une vérification locale de l’interface. Ce palier prouve le rendu des pages, pas une authentification réelle. La configuration finale de l’étape 6 remplacera ce fichier.

7 Étape 3 convertir les pages en EJS

Déplacer les fichiers

Créer backend/views/ et backend/views/partials/. Déplacer public/index.html vers backend/views/login.ejs, public/templates/register.html vers backend/views/register.ejs, et public/templates/agendas.html vers backend/views/agendas.ejs. Garder tout le HTML de chaque page, particulièrement les id utilisés par les scripts. Le dossier public/templates/ n’est plus nécessaire une fois vide.

Corriger les chemins dans les trois vues

Remplacer les références aux ressources par des chemins commençant par / : css/style.css et ../css/style.css deviennent /css/style.css. js/api.js et ../js/api.js deviennent /js/api.js. Même règle pour login.js, register.js et agendas.js.

Remplacer les liens vers index.html et ../index.html par /. Les liens templates/register.html deviennent /register. Les liens agendas.html ou templates/agendas.html deviennent /agendas. Ne pas créer des liens vers /views/agendas.ejs : les vues sont internes au serveur.

Mutualiser le head

Créer backend/views/partials/head.ejs avec ce contenu complet :

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title><%= title %> · ChronoSquad</title>
  <link rel="stylesheet" href="/css/style.css">
</head>
```

Dans chaque vue, remplacer le bloc head entier par l'inclusion ci-dessous. Ne pas conserver un deuxième head autour de l'inclusion.

```
<%- include('partials/head', { title }) %>
```

Conserver les scripts en fin de body et charger api.js avant le script propre à la page. Conserver leur mode actuel de scripts classiques : ne pas ajouter type="module" sans adapter leur partage de la variable api.

Afficher une donnée réellement rendue par EJS

Dans agendas.ejs, remplacer le span vide current-user par :

```
<span id="current-user"><%= user ? user.username : '' %></span>
```

Le contrôleur final transmettra un utilisateur public. L'appel getMe() déjà présent dans agendas.js peut rester : il vérifie aussi la session au chargement et son affectation textContent reste compatible. La liste d'agendas continue d'être construite par agendas.js pour éviter deux rendus concurrents du même composant.

Les trois balises à connaître

- <% instruction %> exécute une condition ou une boucle sans afficher sa valeur.
- <%= valeur %> affiche du texte en échappant les caractères HTML ; utiliser cette forme pour un nom ou une description.
- <%- valeur %> insère du HTML sans échappement ; l'utiliser ici pour include d'une vue partielle maîtrisée, pas pour un texte saisi par un utilisateur.

Exemple pédagogique de boucle, à ne pas ajouter à la liste déjà pilotée par agendas.js sans changer ce dernier :

```
<ul>
  <% for (const agenda of agendas) { %>
    <li><%= agenda.name %></li>
  <% } %>
</ul>
```

Il faut alors transmettre agendas à res.render. EJS ne va pas chercher les données tout seul. Une variable absente peut provoquer une erreur ; transmettre explicitement title et user comme dans les exemples.

Ajouter backend/views/error.ejs

```
<!doctype html>
<html lang="fr">
<%- include('partials/head', { title }) %>
<body>
  <main class="auth">
    <section class="card">
      <h1>Erreur <%= status %></h1>
      <p><%= message %></p>
      <a href="/">Retour à la connexion</a>
    </section>
  </main>
</body>
</html>
```

Lancer npm start et ouvrir http://localhost:3000. Vérifier le titre de page, le CSS, /register, /agendas et une URL inexistante. Dans le code source reçu par le navigateur, les balises EJS doivent avoir disparu. Si vous voyez <% en clair, le modèle a été servi comme fichier au lieu d'être rendu.

8 Étape 4 préparer la persistance et les dépendances métier

```
npm install express-session argon2
```

Argon2 correspond à l'import déjà présent dans User.js. Ne pas ajouter bcryptjs en parallèle sans décider d'une migration des hashes. Les modules node:http, node:fs, node:path et node:crypto sont intégrés à Node ; ils ne s'installent pas avec npm.

Bien distinguer les objets publics des données sauvegardées

User.toJSON() supprime volontairement passwordHash. JSON.stringify(user) appelle automatiquement cette méthode. Passer directement un User à UserRepository.create ferait donc perdre le hash lors de la sauvegarde. Il faut utiliser user.toRecord() pour sauvegarder et user.toJSON() pour répondre au client.

UserRepository.findByUsername renvoie actuellement un objet JSON ordinaire. Il n'a pas la méthode verifyPassword. Il faut reconstruire un User avec les champs du record avant de l'utiliser. Cette reconstruction est incluse dans le contrôleur de connexion proposé plus loin.

Sécuriser les écritures concurrentes avant le vrai mode API

JsonStore fait actuellement un cycle lecture, modification, écriture sans verrou. Deux créations simultanées peuvent lire le même tableau et la dernière écriture efface l'autre ajout. Le client utilise précisément Promise.all pour créer une tâche dans plusieurs agendas : ce problème peut donc arriver avec une seule personne.

Pour un unique processus Node, ajouter une file d'attente partagée par chemin de fichier autour de toute la transaction lecture/modification/écriture. Une file par instance ne suffit pas si plusieurs instances pointent vers le même fichier. Un fichier temporaire puis rename évite en plus de publier un fichier partiellement écrit ; cela ne remplace pas la file d'attente.

Voici une version de JsonStore.js pour les opérations utilisées par le socle. Elle remplace le fichier existant et conserve aussi update et delete. Les fichiers existants sont conservés ; seuls les fichiers absents sont initialisés à [].

```
import fs from 'node:fs/promises';
import path from 'node:path';
```

```
import { fileURLToPath } from 'node:url';
import { randomUUID } from 'node:crypto';

const directory = fileURLToPath(new URL('../../data/', import.meta.url));
const queues = new Map();

export class JsonStore {
  constructor(fileName) {
    this.filePath = path.join(directory, fileName);
  }

  async load() {
    try {
      const value = JSON.parse(await fs.readFile(this.filePath, 'utf8'));
      if (!Array.isArray(value)) throw new Error('Tableau JSON attendu');
      return value;
    } catch (error) {
      if (error.code === 'ENOENT') return [];
      throw error;
    }

  }

  async save(data) {
    await fs.mkdir(directory, { recursive: true });
    const temp = `${this.filePath}.${randomUUID()}.tmp`;
    try {
      await fs.writeFile(temp, JSON.stringify(data, null, 2), 'utf8');
      await fs.rename(temp, this.filePath);
    } finally {
      await fs.rm(temp, { force: true });
    }
  }

  transaction(change) {
    const previous = queues.get(this.filePath) ?? Promise.resolve();
    const current = previous.then(async () => {
      const rows = await this.load();
      const result = await change(rows);
      await this.save(rows);
      return result;
    });
    queues.set(this.filePath, current.catch(() => {}));

    return current;
  }

  async getAll() { return this.load(); }
  async findById(id) {
    return (await this.load()).find((row) => row.id === id);
  }
  insert(record) {
    return this.transaction((rows) => {
      rows.push(record);
      return record;
    });
  }
  update(id, values) {
    return this.transaction((rows) => {
      const index = rows.findIndex((row) => row.id === id);
      if (index < 0) return null;
      rows[index] = { ...rows[index], ...values, id };
      return rows[index];
    });
  }
  delete(id) {

    return this.transaction((rows) => {
      const index = rows.findIndex((row) => row.id === id);
```

```
      if (index < 0) return false;
      rows.splice(index, 1);
      return true;
    });
  }
}
```

Les repositories doivent appeler insert/update/delete/transaction, pas save directement. Ne pas transformer un JSON corrompu en tableau vide : l'erreur doit rester visible pour éviter d'effacer silencieusement des données. Cette file ne coordonne pas plusieurs processus Node ni plusieurs serveurs ; une base transactionnelle sera préférable si l'architecture évolue.

Compléter les repositories

Dans UserRepository.js, remplacer seulement la méthode create existante par cette version. Le test d'unicité se fait dans la même transaction que l'insertion ; deux inscriptions simultanées ne peuvent pas réserver le même nom dans ce processus.

```
async create(record) {
  return this.store.transaction((users) => {
    if (users.some((user) => user.username === record.username)) {
      const error = new Error('Nom déjà pris.');
      error.status = 409;
      throw error;
    }
    users.push(record);
    return record;
  });
}
```

Remplir backend/repositories/AgendaRepository.js :

```
import { JsonStore } from '../storage/JsonStore.js';
export class AgendaRepository {
  constructor() { this.store = new JsonStore('agendas.json'); }
  async findById(id) { return this.store.findById(id); }
  async findByOwner(ownerId) {
    return (await this.store.getAll())
      .filter((agenda) => agenda.ownerId === ownerId);
  }
  async create(record) { return this.store.insert(record); }
}
```

Remplir backend/repositories/EvenementRepository.js :

```
import { JsonStore } from '../storage/JsonStore.js';
export class EvenementRepository {
  constructor() { this.store = new JsonStore('evenements.json'); }
  async findByAgendaIds(ids) {
    const allowed = new Set(ids);
    return (await this.store.getAll())
      .filter((event) => allowed.has(event.agendaId));
  }
  async create(record) { return this.store.insert(record); }
}
```

9 Étape 5 gérer les comptes et la session

Principe de la session

Une connexion réussie crée une session sur le serveur. Le navigateur conserve un cookie contenant un identifiant de session, et le renvoie automatiquement avec les requêtes vers le même site. La session contient userId ; le serveur retrouve ensuite l'utilisateur. Le client ne décide pas de son identité en envoyant un ownerId.

Le code utilise express-session avec son stockage mémoire pour le développement. Un redémarrage déconnecte les utilisateurs mais ne supprime pas leurs comptes ou agendas enregistrés dans data/. Pour conserver les connexions entre redémarrages, il faudra configurer un stockage persistant de sessions. MemoryStore n'est pas une solution de production.

Remplir backend/middleware/RequireAuth.js

```
import { UserRepository } from '../repositories/UserRepository.js';
const users = new UserRepository();

export async function loadUser(req, res, next) {
  const record = req.session.userId
    ? await users.findById(req.session.userId) : null;
  res.locals.user = record ? {
    id: record.id, username: record.username,
    createdAt: record.createdAt
  } : null;
  next();
}

export function requireApiAuth(req, res, next) {
  if (!res.locals.user) {
    return res.status(401).json({ error: 'Non connecté.' });
  }
  next();
}

export function requirePageAuth(req, res, next) {
  if (!res.locals.user) return res.redirect('/');

  next();
}
```

Une page peut rediriger vers la connexion. Une API doit répondre 401 en JSON, car fetch pourrait suivre une redirection et recevoir du HTML à la place des données attendues.

Remplir backend/controllers/UserController.js

```
import argon2 from 'argon2';
import { randomUUID } from 'node:crypto';
import { User } from '../models/User.js';
import { UserRepository } from '../repositories/UserRepository.js';
const users = new UserRepository();

export async function register(req, res) {
  const { username, password } = req.body ?? {};
  if (typeof username !== 'string' || !username.trim() ||
      username.trim().length > 80 || typeof password !== 'string' ||
      password.length < 8 || password.length > 200) {
    return res.status(400).json({
      error: 'Nom requis (80 caractères max), mot de passe de 8 à 200 caractères.'
    });
  }
  const user = new User(randomUUID(), username.trim(),
    await argon2.hash(password));
  await users.create(user.toRecord());
  res.status(201).json(user.toJSON());
}

export function getCurrentUser(req, res) {

  res.json(res.locals.user);
}
```

La limite de mot de passe est un choix de ce socle. Reporter la même règle dans register.js pour que le formulaire donne un message précis. La validation serveur reste nécessaire même si le navigateur valide déjà. Ne pas modifier le mot de passe avec trim(). Les noms sont ici sensibles à la casse après suppression des espaces extérieurs ; documenter cette règle.

Remplir backend/controllers/AuthController.js

```
import { User } from '../models/User.js';
import { UserRepository } from '../repositories/UserRepository.js';
const users = new UserRepository();

export async function login(req, res) {
  const { username, password } = req.body ?? {};
  if (typeof username !== 'string' || typeof password !== 'string' ||
      username.length > 80 || password.length > 200) {
    return res.status(400).json({ error: 'Champs invalides.' });
  }
  const record = await users.findByUsername(username.trim());
  let user = null;
  let valid = false;
  if (record) {
    user = new User(record.id, record.username,
      record.passwordHash, record.createdAt);
    valid = await user.verifyPassword(password);
  }
  if (!valid) {
    return res.status(401).json({ error: 'Identifiants invalides.' });
  }
  await new Promise((resolve, reject) => {

    req.session.regenerate((err) => err ? reject(err) : resolve());
  });
  req.session.userId = user.id;
  await new Promise((resolve, reject) => {
    req.session.save((err) => err ? reject(err) : resolve());
  });
  res.json(user.toJSON());
}
```

```
export async function logout(req, res) {
  await new Promise((resolve, reject) => {
    req.session.destroy((err) => err ? reject(err) : resolve());
  });
  res.clearCookie('chronosquad.sid', { path: '/' });
  res.status(204).end();
}
```

La régénération évite de conserver l'identifiant de session précédant la connexion. save est attendu avant la réponse pour que la navigation suivante retrouve la session. Un hash d'exemple mal formé peut déclencher une erreur interne ; tester avec un compte créé par register.

Relation avec l'authentification du cours

Les pages 62 à 69 du CM06 présentent un JWT placé dans un cookie et vérifié avant d'alimenter res.locals.user. Le principe commun est de vérifier l'identité avant les routes protégées. Le présent guide remplace ce mécanisme par une session serveur, déjà prévue dans votre conception. Ne pas empiler JWT et express-session pour le même besoin.

Un JWT signé n'est pas nécessairement chiffré. Le hachage côté navigateur illustré dans le cours ne remplace pas HTTPS et un hash de mot de passe adapté côté serveur. Ici, le modèle existant est conservé avec Argon2. Pour la démonstration HTTP en réseau local, utiliser uniquement des comptes de test ; une exploitation réelle nécessite HTTPS et une configuration de sécurité adaptée.

10 Étape 6 brancher les routes et la configuration finale

À partir d'ici, remplacer le backend/app.js provisoire de l'étape 2 par le fichier suivant, après avoir créé tous les fichiers importés dans cette étape et la suivante. Ne pas conserver en parallèle les routes provisoires non protégées.

backend/app.js final

```
import express from 'express';
import session from 'express-session';
import { randomBytes } from 'node:crypto';
import { fileURLToPath } from 'node:url';
import { loadUser } from './middleware/RequireAuth.js';
import { pageRouter } from './routes/PageRoutes.js';
import { apiRouter } from './routes/Routes.js';

export const app = express();
const localPath = (value) =>
  fileURLToPath(new URL(value, import.meta.url));
const production = process.env.NODE_ENV === 'production';
if (production && !process.env.SESSION_SECRET) {
  throw new Error('SESSION_SECRET doit être configuré.');
}
const secret = process.env.SESSION_SECRET ??
  randomBytes(32).toString('hex');

app.set('views', localPath('./views/'));
app.set('view engine', 'ejs');
app.use('/css', express.static(localPath('../public/css/')));
app.use('/js', express.static(localPath('../public/js/')));

app.use(express.json({ limit: '100kb' }));
app.use(express.urlencoded({ extended: false, limit: '100kb' }));
app.use(session({
  name: 'chronosquad.sid', secret,
  resave: false, saveUninitialized: false,
  cookie: {
```

```
    httpOnly: true, sameSite: 'lax', secure: production,
    maxAge: 8 * 60 * 60 * 1000
  }
}));
app.use(loadUser);
app.use((req, res, next) => {
  res.set('Cache-Control', 'no-store');
  next();
});
app.use('/api', apiRouter);
app.use('/', pageRouter);

app.use((req, res) => {
  res.status(404).render('error', {
    title: 'Page introuvable', status: 404,
    message: 'Cette page n’existe pas.'

  });
});
app.use((err, req, res, next) => {
  if (res.headersSent) return next(err);
  console.error(err);
  const status = Number.isInteger(err.status) &&
    err.status >= 400 && err.status <= 599 ? err.status : 500;
  const message = status >= 500 ? 'Erreur interne du serveur.' :
    (status === 400 ? 'Requête invalide.' : err.message);
  const apiRequest = req.originalUrl.split('?')[0];
  if (apiRequest === '/api' || apiRequest.startsWith('/api/')) {
    return res.status(status).json({ error: message });
  }
  res.status(status).render('error', {
    title: 'Erreur', status, message
  });
});
```

L’ordre est intentionnel : ressources publiques, analyse des corps, session, utilisateur, routes et erreurs. express.json n’autorise ni ne valide les données ; il les décode. express.urlencoded sert aux formulaires HTML classiques ; le client actuel utilise JSON avec fetch. Le middleware d’erreur conserve bien quatre paramètres, dont next.

La 404 de l’API sera installée dans son routeur pour ne jamais renvoyer la page d’erreur HTML à api.js. Express 5 transmet les rejets des fonctions async aux gestionnaires d’erreurs. Une erreur lancée dans un callback de timer détaché doit encore être transmise explicitement à next.

Le secret aléatoire de développement permet de respecter npm install puis npm start sans fichier secret obligatoire. Pour le fixer dans PowerShell avant de lancer le serveur :

```
$env:SESSION_SECRET = node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"
npm start
```

Cette variable n’est valable que pour le terminal courant et ses processus enfants. Un fichier .env n’est pas lu automatiquement par ce code ; si l’équipe choisit dotenv, il faut l’installer et le charger explicitement avant l’utilisation des variables. Ne jamais versionner un vrai secret. Fixer le secret ne rend pas MemoryStore persistant.

Créer backend/routes/PageRoutes.js

```
import { Router } from 'express';
import { requirePageAuth } from '../middleware/RequireAuth.js';
export const pageRouter = Router();

pageRouter.get('/', (req, res) => {
  if (res.locals.user) return res.redirect('/agendas');
  res.render('login', { title: 'Connexion' });
});
pageRouter.get('/register', (req, res) => {
  res.render('register', { title: 'Créer un compte' });
});
pageRouter.get('/agendas', requirePageAuth, (req, res) => {
  res.render('agendas', {
    title: 'Mes agendas', user: res.locals.user
  });
});
```

Remplir backend/routes/Routes.js

```
import { Router } from 'express';
import { requireApiAuth } from '../middleware/RequireAuth.js';
import * as users from '../controllers/UserController.js';
import * as auth from '../controllers/AuthController.js';
import * as agendas from '../controllers/AgendaController.js';
import * as events from '../controllers/EvenementController.js';
export const apiRouter = Router();

apiRouter.post('/users', users.register);
apiRouter.post('/auth/login', auth.login);
apiRouter.post('/auth/logout', requireApiAuth, auth.logout);
apiRouter.get('/users/me', requireApiAuth, users.getCurrentUser);
apiRouter.get('/agendas', requireApiAuth, agendas.getAgendas);
apiRouter.get('/agendas/:id', requireApiAuth, agendas.getAgenda);
apiRouter.post('/agendas', requireApiAuth, agendas.createAgenda);
apiRouter.get('/evenements', requireApiAuth, events.getEvenements);
apiRouter.post('/agendas/:agendaId/evenements',
  requireApiAuth, events.createEvenement);
apiRouter.use((req, res) => {
  res.status(404).json({ error: 'Route API introuvable.' });
});
```

Le préfixe /api est déjà posé par app.use('/api', apiRouter). Le répéter dans ce routeur donnerait /api/api/agendas. Les fonctions sont exportées par leur nom ; les importations ci-dessus doivent correspondre exactement, y compris les majuscules des fichiers.

11 Étape 7 compléter les agendas et événements

Remplir backend/controllers/AgendaController.js

```
import { randomUUID } from 'node:crypto';
import { Agenda } from '../models/Agenda.js';
import { AgendaRepository } from '../repositories/AgendaRepository.js';
const agendas = new AgendaRepository();

export async function getAgendas(req, res) {
  res.json(await agendas.findByOwner(res.locals.user.id));
}

export async function getAgenda(req, res) {
  const agenda = await agendas.findById(req.params.id);
  if (!agenda || agenda.ownerId !== res.locals.user.id) {
    return res.status(404).json({ error: 'Agenda introuvable.' });
  }
  res.json(agenda);
}

export async function createAgenda(req, res) {
  const { name, color = '#3b82f6' } = req.body ?? {};
  if (typeof name !== 'string' || !name.trim() || name.length > 100 ||
      typeof color !== 'string' || !/^#[0-9a-fA-F]{6}$/.test(color)) {
    return res.status(400).json({ error: 'Nom ou couleur invalide.' });

  }
  const agenda = new Agenda(randomUUID(), name,
    res.locals.user.id, color);
  const record = agenda.toJSON();
  await agendas.create(record);
  res.status(201).json(record);
}
```

L'ordre du constructeur est celui de votre classe : id, name, ownerId, color. Il ne faut pas confondre ownerId et color. Le serveur attribue le propriétaire ; même si le client envoie un autre ownerId, il ne doit pas être utilisé.

Remplir backend/controllers/EvenementController.js

```
import { randomUUID } from 'node:crypto';
import { Evenement } from '../models/Evenement.js';
import { AgendaRepository } from '../repositories/AgendaRepository.js';
import { EvenementRepository } from '../repositories/EvenementRepository.js';
const agendas = new AgendaRepository();
const events = new EvenementRepository();

export async function getEvenements(req, res) {
  const own = await agendas.findByOwner(res.locals.user.id);
  res.json(await events.findByAgendaIds(own.map((agenda) => agenda.id)));
}

export async function createEvenement(req, res) {
  const agenda = await agendas.findById(req.params.agendaId);
  if (!agenda || agenda.ownerId !== res.locals.user.id) {
    return res.status(404).json({ error: 'Agenda introuvable.' });
  }
  const { title, start, end, description = '', location = '' } =
    req.body ?? {};
  const startDate = new Date(start);
  const endDate = new Date(end);
  if (typeof title !== 'string' || !title.trim() || title.length > 200 ||

      typeof start !== 'string' || typeof end !== 'string' ||
      !Number.isFinite(startDate.getTime()) ||
```

```
      !Number.isFinite(endDate.getTime()) || endDate <= startDate ||
      typeof description !== 'string' || description.length > 5000 ||
      typeof location !== 'string' || location.length > 200) {
    return res.status(400).json({ error: 'Événement invalide.' });
  }
  const event = new Evenement(randomUUID(), agenda.id, title,
    start, end, description, location);
  const record = event.toJSON();
  await events.create(record);
  res.status(201).json(record);
}
```

Votre constructeur Evenement attend description avant location. Les dates sont sérialisées en ISO par JSON.stringify. Conserver la conversion ISO déjà faite côté client et vérifier les heures affichées après rechargement. Les autorisations sont vérifiées à chaque requête ; masquer un bouton dans la page ne protège pas une donnée.

Ce socle traite uniquement les agendas possédés. Le partage prévu dans le sujet nécessitera une règle d’accès plus riche, réutilisée aussi bien pour les pages que pour toutes les opérations API.

12 Étape 8 connecter définitivement le navigateur

Remplacer les redirections

Dans public/js/login.js, remplacer les deux occurrences de templates/agendas.html par /agendas : la redirection après getMe et celle après login. Dans public/js/register.js, remplacer ../index.html par /. Dans public/js/agendas.js, modifier goToLogin pour utiliser /.

```
// login.js, dans les deux redirections
window.location.href = '/agendas';

// register.js et fonction goToLogin de agendas.js
window.location.href = '/';
```

Ces extraits indiquent les remplacements à faire ; ne pas les coller comme des instructions indépendantes en tête des scripts, ce qui provoquerait une navigation immédiate.

Activer realApi

Dans public/js/api.js, modifier uniquement la constante :

```
const USE_MOCK = false;
```

Ne pas recopier l’API dans chaque script. request() centralise déjà fetch, l’envoi JSON, les erreurs et le cas 204. credentials: 'same-origin' transmet le cookie vers le même serveur. Les URL /api/... fonctionnent aussi depuis un autre ordinateur, car elles désignent le serveur qui a fourni la page.

Les comptes mock ne sont pas transférés automatiquement dans data/users.json. Créer de nouveaux comptes de test dans la véritable API. Une session mock ne peut pas ouvrir la page /agendas protégée par le serveur. Les anciennes données locales peuvent être supprimées séparément dans les outils du navigateur lorsqu’elles ne sont plus utiles.

Un renforcement utile de request() consiste à lever une erreur explicite lorsqu’une réponse réussie, hors 204, ne contient pas le JSON attendu. Le code actuel transforme une réponse HTML imprévue en null et peut provoquer une erreur plus loin dans agendas.js.

EJS et formulaires classiques

Vos formulaires actuels annulent submit avec preventDefault et appellent api.login ou api.register. Cette stratégie fonctionne avec EJS et permet de garder les messages d'erreur existants.

Dans une variante plus proche des formulaires du cours, form aurait action="/login" et method="POST" ; Express lirait req.body grâce à express.urlencoded, puis ferait res.redirect ou res.render avec un message. Il faudrait alors adapter le script qui intercepte submit et créer cette route de page POST. Ne pas mélanger cette variante avec l'API JSON sans définir précisément le comportement attendu.

13 Étape 9 vérifier le fonctionnement réel

Premier démarrage

```
npm install
npm start
```

Ces deux commandes successives sont équivalentes au npm install && npm start du sujet. Dans Windows PowerShell ancien, && peut ne pas être accepté ; PowerShell 7 le permet. Le terminal doit annoncer le port 3000 et rester actif.

Ouvrir les pages via http://localhost:3000, pas en double-cliquant sur un fichier, pas avec Live Server sur un autre port. Toutes les ressources et l'API doivent venir de la même origine ; aucun réglage CORS n'est nécessaire pour cette architecture.

Vérifications à faire dans cet ordre

1. GET / renvoie 200 et du HTML ; /css/style.css et /js/api.js renvoient 200. Le titre vient de la variable EJS et aucune balise <% n'est visible dans la source reçue.

2. Sans connexion, /agendas redirige vers / ; GET /api/users/me renvoie 401 JSON.

3. Créer un compte neuf avec un mot de passe de test d'au moins huit caractères. Attendre 201 ; vérifier que users.json contient passwordHash et qu'aucune réponse API ne le contient.

4. Créer le même nom une seconde fois : attendre 409. Essayer des champs invalides : attendre 400 sans nouvelle entrée.

5. Essayer un mauvais mot de passe : 401. Se connecter correctement : 200, cookie chronosquad.sid et accès à /agendas.

6. Créer deux agendas puis une tâche dans chacun. Attendre des réponses 201 et des enregistrements avec les bons identifiants.

7. Créer une tâche dans plusieurs agendas depuis la feuille de création. Vérifier que tous les événements restent dans evenements.json après les requêtes parallèles.

8. Recharger la page : les agendas et événements doivent revenir par l'API. Vérifier aussi les heures et couleurs.

9. Se déconnecter : 204, puis /api/users/me doit renvoyer 401 et /agendas rediriger.

10. Arrêter et relancer le serveur. Se reconnecter : les données doivent toujours être présentes. Avec MemoryStore, devoir se reconnecter est attendu.

11. Ouvrir une fenêtre privée et créer un second utilisateur. Il ne doit pas lire ni alimenter un agenda du premier, même en recopiant son identifiant dans l'URL.

12. Tester /inconnue et /api/inconnue : 404 HTML pour la première, 404 JSON pour la seconde. Un JSON mal formé envoyé à l'API doit donner 400, pas arrêter le processus.

Les outils de développement du navigateur, onglet Réseau, permettent de vérifier méthode, URL, statut et corps de chaque requête. Une interface qui affiche une donnée sans requête /api ne prouve pas que le mode mock a été désactivé.

Tests automatisés à ajouter ensuite

Les trois fichiers de tests actuels sont vides. Prévoir des tests utiles : sérialisation User.toRecord et User.toJSON, deux insertions simultanées dans un même fichier, noms d'utilisateur uniques, dates invalides, accès interdit aux agendas d'autrui et conservation des données après redémarrage. Exécuter ces tests sur un répertoire temporaire ou une copie des données, jamais en réinitialisant les fichiers de travail de l'équipe.

Le découpage app.js / server.js permet d'importer l'application dans un test sans démarrer automatiquement le port 3000. Les exemples de ce guide doivent être testés dans la branche de migration avant intégration : ils constituent une proposition de code, pas une migration déjà appliquée et validée dans votre dépôt.

14 Configuration réseau et dépannage

Accéder depuis un autre ordinateur

Un membre démarre le serveur sur sa machine. L'écoute sur 0.0.0.0 accepte les connexions IPv4 sur les interfaces réseau disponibles. Dans le terminal Windows, ipconfig permet de repérer l'IPv4 de la carte utilisée. Depuis un autre ordinateur du même réseau, ouvrir http://ADRESSE_IP_DU_SERVEUR:3000.

localhost désigne toujours l'ordinateur sur lequel le navigateur tourne. Sur le deuxième ordinateur, localhost:3000 ne désigne donc pas le premier. Garder les fetch sous la forme /api/... évite de coder en dur une adresse locale erronée.

Si la page ne répond pas à distance, vérifier dans l'ordre : serveur toujours actif, bonne IPv4, même réseau, port 3000 autorisé par le pare-feu sur le réseau privé, absence d'isolation des clients Wi-Fi. Il ne faut pas désactiver complètement le pare-feu. Le sujet ne nécessite pas de publier le serveur sur Internet.

Erreurs fréquentes et leur cause

- ERR_MODULE_NOT_FOUND pour express ou ejs : lancer npm install dans le dossier contenant le bon package.json. Pour argon2, vérifier son installation et les messages de compatibilité de la version Node utilisée.
- EADDRINUSE : un processus écoute déjà sur 3000. Arrêter l'ancien serveur ou identifier le processus avec Get-NetTCPConnection -LocalPort 3000 ; ne pas modifier arbitrairement le port de rendu.
- Failed to lookup view : vérifier backend/views/, le nom exact du fichier et app.set('views'). res.render('login') recherche login.ejs, pas public/index.html.
- user is not defined dans EJS : transmettre user ou utiliser une valeur explicitement initialisée. Le palier provisoire transmet null ; le palier final utilise res.locals.user.
- req.body absent : vérifier express.json placé avant les routes et l'en-tête Content-Type: application/json. Valider ensuite les champs : le parser ne le fait pas.
- CSS ou JS introuvables : utiliser /css/style.css et /js/api.js, sans ../ et sans /public dans l'URL.
- / affiche encore l'ancien HTML : vérifier qu'aucun express.static(public) avec index par défaut ne répond avant la route de page. La configuration de ce guide monte seulement /css et /js.

- Requête qui attend indéfiniment : vérifier qu'un middleware répond ou appelle next(). Après res.json, utiliser return si du code pourrait encore répondre.
- Connexion réussie mais page refusée : vérifier le cookie, la sauvegarde de session et l'utilisation d'un hôte cohérent. localhost et l'adresse IP ne partagent pas nécessairement le même cookie.
- Cookie absent en HTTP : ne pas activer secure: true dans le contexte local HTTP. La configuration proposée l'active lorsque NODE_ENV vaut production, où HTTPS doit être prévu.
- Changement de compte entre navigateurs impossible avec le mock : c'est attendu ; les stockages locaux sont séparés. Vérifier USE_MOCK = false et les vraies requêtes réseau.
- Données qui disparaissent après des créations simultanées : vérifier que toutes les écritures passent par la même file transactionnelle par fichier, et qu'un seul processus serveur utilise ces JSON.

15 Travail restant et ordre conseillé pour l'équipe

Liste des fichiers à revoir

- package.json et package-lock.json : dépendances réelles, scripts et versions partagées.
- backend/server.js : seul point d'écoute ; backend/app.js : configuration centrale.
- backend/views/ : trois vues converties, un head commun et une page d'erreur.
- backend/routes/PageRoutes.js et Routes.js : distinction HTML / JSON.
- backend/middleware/RequireAuth.js : identité commune et protections adaptées.
- Les quatre contrôleurs : opérations déjà demandées par realApi.
- Les trois repositories et JsonStore : données persistantes et écritures coordonnées.
- public/js/api.js : mode réel ; login.js, register.js et agendas.js : chemins de navigation.
- README.md et docs/sprint0/conception.md : architecture réelle, routes réelles, Argon2, stockage de session et limites.
- backend/tests/ : remplacer les fichiers vides par des tests ciblés.

Ordre de réalisation proposé

Faire un premier ensemble cohérent avec Express, les vues EJS et les ressources statiques. Le vérifier avant de continuer. Ajouter ensuite stockage, comptes et session. Puis brancher les routes d'agendas et d'événements. Activer enfin realApi, effectuer la recette complète et actualiser la documentation.

Pour travailler à plusieurs, fixer d'abord les URL, le format JSON et les noms des fonctions exportées. Une personne peut traiter les vues, une autre les comptes, une autre les données et une autre la recette. Les changements qui touchent app.js ou les contrats partagés doivent être coordonnés pour éviter des assemblages incompatibles.

Les fonctionnalités du sujet à ne pas oublier

La migration décrite fournit la base du serveur et les opérations actuelles du client. Elle ne termine pas l'application du sujet. Il reste notamment à ajouter modification et suppression des rendez-vous : choisir des routes telles que PUT /api/evenements/:id et DELETE /api/evenements/:id, vérifier le propriétaire de l'agenda, valider les champs, compléter le repository et ajouter les actions client.

L’affichage simultané de plusieurs agendas nécessite de remplacer la sélection unique de la semaine par un ensemble d’identifiants sélectionnés, puis de filtrer les événements sur cet ensemble. Les fonctionnalités de récurrence, recherche, partage et import/export viennent après le client minimal. Elles utilisent les mêmes couches Express, contrôleurs et repositories.

La création d’un événement dans plusieurs agendas produit actuellement plusieurs enregistrements indépendants. Même avec des écritures coordonnées, l’ensemble n’est pas une transaction globale : si une requête échoue, certaines créations peuvent avoir réussi. Définir le comportement attendu et afficher les succès/échecs partiels, ou créer plus tard une route de création groupée atomique.

Avant un usage autre qu’une démonstration locale, compléter la gestion des sessions persistantes, HTTPS, la protection des opérations sensibles contre les requêtes indésirables, les limites de tentatives de connexion et les sauvegardes. Le paramètre production du code ne réalise pas ces travaux à lui seul.

16 Repères de lecture et références

Documents fournis

projet (2).pdf, page 1 : serveur unique, routes distinctes, port 3000, lancement, persistance, accès réseau et fonctionnalités. Les exigences du sujet sont distinguées des choix de mise en œuvre dans le préambule.

2026_2027_acl_js_cm_05 (1).pdf : pages 3 à 8 pour Node, npm et package.json ; pages 9 à 13 pour les fichiers ; pages 15 à 28 pour HTTP et les requêtes ; pages 29 à 33 pour fetch et CORS ; pages 34 à 46 pour les formulaires ; pages 48 à 57 pour les échanges de données et JSON.

2026_2027_acl_js_cm_06.pdf : pages 3 à 9 pour Express et les routes ; pages 10 à 18 pour middleware et erreurs ; pages 19 à 26 pour fichiers statiques et parsers ; pages 30 à 35 pour séparation serveur, application, contrôleurs et vues ; pages 40 à 48 pour EJS et CSS ; pages 51 à 53 pour res.locals et formulaires ; pages 62 à 69 pour JWT, cookies et utilisateur courant.

Documentation officielle complémentaire

- Express 5 et différences de version : https://expressjs.com/en/guide/migrating-5/
- EJS, balises et inclusions : https://ejs.co/
- express-session, cookies et limites du stockage mémoire : https://expressjs.com/en/resources/middleware/session/
- Versions maintenues de Node.js : https://nodejs.org/en/about/previous-releases

Les exemples d’architecture et de migration sont adaptés à votre dépôt. Ils conservent les modules ES et les chemins réels de sprint0. Les pages citées sont les numéros de pages PDF, comptés à partir de la première page.